

BÀI THỰC HÀNH DEEP LEARNING CHO KHOA HỌC DỮ LIỆU

Lab 5. Mạng Hồi Quy: RNN, LSTM, GRU ứng dụng Xử Lý Ngôn Ngữ Tự Nhiên Tiếng Việt

Hà Minh Tuấn

Khoa Toán - Tin học, ĐHKHTN, ĐHQG-HCM

Ngày 7 tháng 4 năm 2026

Bài Thực Hành

Mạng Nơ-ron Hồi Quy (RNN)

Dự Đoán Chuỗi Thời Gian

Môn học: Deep Learning — Học kỳ 2025–2026

Mục tiêu tổng quát. Sinh viên xây dựng, huấn luyện và đánh giá mô hình RNN cho bài toán dự đoán chuỗi thời gian (nhiệt độ). Rèn luyện kỹ năng chia dữ liệu, vẽ đường cong học tập, điều chỉnh siêu tham số, giám sát gradient bằng TensorBoard, phát hiện và khắc phục overfitting, underfitting, vanishing/exploding gradient.

Mục lục

I	Nền Tảng Và Thu Thập Dữ Liệu	4
1	Ôn tập lý thuyết RNN	4
1.1	Cấu trúc RNN	4
1.2	Vấn đề Vanishing/Exploding Gradient	4
1.3	Ứng dụng cho chuỗi thời gian	4
2	Thiết lập môi trường	4
3	Lấy dữ liệu từ API	5
3.1	Giới thiệu Open-Meteo API	5
3.2	Khám phá dữ liệu	6

II	Tiền Xử Lý Và Xây Dựng Mô Hình	6
4	Tiền xử lý dữ liệu chuỗi thời gian	7
4.1	Chuẩn hóa, tạo cửa sổ trượt, chia tập dữ liệu	7
5	Định nghĩa mô hình RNN	9
6	Vòng lặp huấn luyện	10
7	Huấn luyện và vẽ đường cong học tập	11
III	Điều Chỉnh Siêu Tham Số	12
8	Thí nghiệm 1 — Hidden Size	12
9	Thí nghiệm 2 — Learning Rate	13
10	Thí nghiệm 3 — Sequence Length	13
11	Thí nghiệm 4 — Số lớp (Num Layers)	13
12	Tổng hợp kết quả siêu tham số	14
IV	Giám Sát Quá Trình Huấn Luyện Bằng TensorBoard	14
13	Tích hợp TensorBoard vào training loop	14
13.1	Nguyên lý	14
13.2	Code	15
14	Chạy nhiều thí nghiệm với TensorBoard	16
15	Hướng dẫn đọc kết quả TensorBoard	17
V	Phát Hiện Và Khắc Phục Vấn Đề	18
16	Giám sát gradient trực tiếp	18
17	Kỹ thuật 1 — Gradient Clipping	20
17.1	Lý thuyết	20
17.2	Thực hành	20
18	Kỹ thuật 2 — Dropout	20
18.1	Lý thuyết	20
18.2	Thực hành	21
19	Kỹ thuật 3 — Weight Decay (L2 Regularization)	21
19.1	Lý thuyết	21
20	Kỹ thuật 4 — Khởi tạo trọng số	22
20.1	Lý thuyết	22

21 Kỹ thuật 5 — LayerNorm	22
22 Bảng tổng hợp các kỹ thuật	23
VI Bài Tập	23
23 Cấp độ 1 — Gõ lại code và nhận xét	23
23.1 Yêu cầu chi tiết	23
23.2 Hình thức nộp	24
24 Cấp độ 2 — Tự code theo yêu cầu	24
24.1 Đề bài	24
24.2 Yêu cầu bắt buộc (6 điểm)	25
24.3 Yêu cầu nâng cao (4 điểm)	25
24.4 Gợi ý cấu trúc code (chỉ có khung, sinh viên tự viết nội dung)	25
24.5 Quy định	26
VII Tài Liệu Tham Khảo	26

Phần I

Nền Tảng Và Thu Thập Dữ Liệu

1 Ôn tập lý thuyết RNN

1.1 Cấu trúc RNN

Mạng nơ-ron hồi quy (Recurrent Neural Network) xử lý dữ liệu tuần tự bằng cách duy trì một trạng thái ẩn h_t qua từng bước thời gian:

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h) \quad (1)$$

$$\hat{y}_t = W_{hy} h_t + b_y \quad (2)$$

Trong đó:

- $x_t \in \mathbb{R}^d$: đầu vào tại bước t (d là số features).
- $h_t \in \mathbb{R}^n$: trạng thái ẩn (n là `hidden_size`).
- $W_{xh} \in \mathbb{R}^{n \times d}$, $W_{hh} \in \mathbb{R}^{n \times n}$: ma trận trọng số.
- $\hat{y}_t$: giá trị dự đoán tại bước t .

1.2 Vấn đề Vanishing/Exploding Gradient

Khi lan truyền ngược qua T bước, gradient tích lũy qua phép nhân ma trận:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_T} \prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}}$$

Nếu $\|W_{hh}\| < 1$ liên tục: gradient $\rightarrow 0$ (**vanishing**) — mô hình không học được thông tin xa. Nếu $\|W_{hh}\| > 1$ liên tục: gradient $\rightarrow \infty$ (**exploding**) — loss bùng nổ thành NaN.

1.3 Ứng dụng cho chuỗi thời gian

Với chuỗi $[x_1, x_2, \dots, x_T]$, ta dùng cửa sổ trượt (sliding window) kích thước w :

- **Đầu vào:** $[x_t, x_{t+1}, \dots, x_{t+w-1}]$ — w giá trị quá khứ.
- **Đầu ra:** x_{t+w} — giá trị tiếp theo cần dự đoán.

RNN đọc tuần tự w giá trị, cập nhật h_t qua mỗi bước, rồi dùng h_T để dự đoán $\hat{y}$.

2 Thiết lập môi trường

```
# === CÀI ĐẶT THU VIÊN ===
# !pip install torch numpy pandas matplotlib scikit-learn requests tensorboard

import torch
import torch.nn as nn
import numpy as np
```

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
from torch.utils.tensorboard import SummaryWriter
import warnings
warnings.filterwarnings('ignore')

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Thiết bị: {device}")

# Dạy seed để kết quả có thể tái lập
def set_seed(seed=42):
    torch.manual_seed(seed)
    np.random.seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

set_seed(42)

```

3 Lấy dữ liệu từ API

3.1 Giới thiệu Open-Meteo API

Open-Meteo là API thời tiết miễn phí, không cần API key. Chúng ta lấy nhiệt độ trung bình hàng ngày của TP. Hồ Chí Minh trong 3 năm (khoảng 1095 điểm dữ liệu).

```

import requests

def fetch_weather_data():
    """
    Gọi Open-Meteo API để lấy nhiệt độ trung bình hàng ngày.
    Trả về DataFrame gồm cột 'date' và 'temperature'.

    Địa điểm: TP. Hồ Chí Minh (lat=10.82, lon=106.63)
    Khoảng thời gian: 3 năm gần nhất
    """
    url = "https://archive-api.open-meteo.com/v1/archive"
    params = {
        "latitude": 10.82,
        "longitude": 106.63,
        "start_date": "2022-01-01",
        "end_date": "2024-12-31",
        "daily": "temperature_2m_mean",
        "timezone": "Asia/Ho_Chi_Minh"
    }

    response = requests.get(url, params=params)

    if response.status_code != 200:
        raise ConnectionError(f"API trả về mã lỗi: {response.status_code}")

    data = response.json()

```

```

df = pd.DataFrame({
    'date': pd.to_datetime(data['daily']['time']),
    'temperature': data['daily']['temperature_2m_mean']
})

# Kiểm tra dữ liệu thiếu
missing = df['temperature'].isna().sum()
if missing > 0:
    print(f"Canh bao: {missing} ngày thiếu dữ liệu -> diện băng nơi suy")
    df['temperature'] = df['temperature'].interpolate(method='linear')

return df

df = fetch_weather_data()

print(f"So diem du lieu : {len(df)}")
print(f"Khoang thoi gian: {df['date'].iloc[0].date()} den
↪ {df['date'].iloc[-1].date()}")
print(f"Nhiệt độ (°C) : min={df['temperature'].min():.1f}, "
      f"max={df['temperature'].max():.1f}, "
      f"mean={df['temperature'].mean():.1f}")

```

3.2 Khám phá dữ liệu

```

fig, axes = plt.subplots(1, 2, figsize=(15, 4))

axes[0].plot(df['date'], df['temperature'], linewidth=0.6, color='steelblue')
axes[0].set_title('Nhiệt độ trung bình hàng ngày - TP.HCM')
axes[0].set_ylabel('°C')
axes[0].grid(True, alpha=0.3)

axes[1].hist(df['temperature'], bins=40, color='coral', edgecolor='white')
axes[1].set_title('Phân phối nhiệt độ')
axes[1].set_xlabel('°C')
axes[1].set_ylabel('Tan suất')

plt.tight_layout()
plt.savefig('01_kham pha.png', dpi=150, bbox_inches='tight')
plt.show()

```

Câu hỏi 1. Dữ liệu có tính mùa vụ (seasonality) không? Chu kỳ bao lâu? Điều này gợi ý gì về `sequence_length` nên chọn?

Phần II

Tiền Xử Lý Và Xây Dựng Mô Hình

4 Tiền xử lý dữ liệu chuỗi thời gian

4.1 Chuẩn hóa, tạo cửa sổ trượt, chia tập dữ liệu

```
class TimeSeriesProcessor:
    """
    Lớp tiền xử lý dữ liệu chuỗi thời gian.

    Pipeline:
        Giá trị gốc -> Chuẩn hóa [0,1] -> Cửa sổ trượt -> Chia train/val/test

    Lưu ý quan trọng:
        - Chuẩn hóa FIT trên tập train, TRANSFORM trên val/test
          (tránh data leakage).
        - Chia theo thu tu thời gian, KHÔNG shuffle
          (dữ liệu tương lai không được lo vào quá khứ).
    """

    def __init__(self, seq_len=30):
        self.seq_len = seq_len
        self.scaler = MinMaxScaler(feature_range=(0, 1))

    def fit_transform(self, values):
        """Fit scaler trên dữ liệu và chuyển đổi."""
        v = np.array(values, dtype=np.float32).reshape(-1, 1)
        return self.scaler.fit_transform(v).flatten()

    def transform(self, values):
        """Chỉ transform (dùng cho val/test)."""
        v = np.array(values, dtype=np.float32).reshape(-1, 1)
        return self.scaler.transform(v).flatten()

    def inverse(self, values):
        """Chuyển về giá trị gốc."""
        return self.scaler.inverse_transform(
            np.array(values).reshape(-1, 1)
        ).flatten()

    def create_sequences(self, data):
        """
        Tạo cặp (X, y) bằng cửa sổ trượt.
        X: (N, seq_len, 1) - đầu vào cho RNN
        y: (N,) - giá trị cần dự đoán
        """
        X, y = [], []
        for i in range(len(data) - self.seq_len):
            X.append(data[i : i + self.seq_len])
            y.append(data[i + self.seq_len])
```

```

X = np.array(X, dtype=np.float32).reshape(-1, self.seq_len, 1)
y = np.array(y, dtype=np.float32)
return X, y

def split(self, X, y, train_ratio=0.7, val_ratio=0.15):
    """
    Chia du lieu theo thu tu thoi gian:
    /--- 70% train ---/--- 15% val ---/--- 15% test ---/
    """
    n = len(X)
    t1 = int(n * train_ratio)
    t2 = int(n * (train_ratio + val_ratio))

    splits = {
        'train': (X[:t1], y[:t1]),
        'val': (X[t1:t2], y[t1:t2]),
        'test': (X[t2:], y[t2:]),
    }
    for name, (sx, sy) in splits.items():
        print(f" {name:5s}: {len(sx):5d} mau")
    return splits

# === AP DUNG ===
processor = TimeSeriesProcessor(seq_len=30)

# Chi fit scaler tren 70% dau (phan train)
train_end = int(len(df) * 0.7)
raw = df['temperature'].values

scaled_train = processor.fit_transform(raw[:train_end])
scaled_rest = processor.transform(raw[train_end:])
scaled_all = np.concatenate([scaled_train, scaled_rest])

X_all, y_all = processor.create_sequences(scaled_all)
splits = processor.split(X_all, y_all)

# Chuyen sang tensor
def to_tensor(X, y):
    return (torch.FloatTensor(X).to(device),
            torch.FloatTensor(y).to(device))

Xtr, ytr = to_tensor(*splits['train'])
Xva, yva = to_tensor(*splits['val'])
Xte, yte = to_tensor(*splits['test'])

print(f"\nShape: X_train={Xtr.shape}, y_train={ytr.shape}")

```

Câu hỏi 2. Tại sao scaler chỉ được fit trên phần train? Nếu fit trên toàn bộ dữ liệu thì xảy ra vấn đề gì?

Câu hỏi 3. Tại sao chia dữ liệu theo thứ tự thời gian mà không shuffle ngẫu nhiên?

5 Định nghĩa mô hình RNN

```
class SimpleRNN(nn.Module):
    """
    Mô hình RNN cơ bản cho dự đoán chuỗi thời gian.

    Kiến trúc:
        Input (batch, seq_len, 1)
        |
        nn.RNN (có thể nhiều lớp chồng)
        |
        Hidden cuối cùng (batch, hidden_size)
        |
        nn.Linear -> Output (batch, 1)

    Tham số:
        input_size : số features (=1 cho 1 biến)
        hidden_size : số neuron lớp ẩn
        num_layers : số lớp RNN chồng lên nhau
        dropout : tỉ lệ dropout giữa các lớp (chỉ khi num_layers > 1)
    """

    def __init__(self, input_size=1, hidden_size=64,
                  num_layers=1, dropout=0.0):
        super().__init__()

        self.rnn = nn.RNN(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout if num_layers > 1 else 0
        )
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        """
        x : (batch, seq_len, input_size)
        -> : (batch,)
        """
        out, _ = self.rnn(x)          # out: (batch, seq_len, hidden)
        last = out[:, -1, :]          # (batch, hidden)
        pred = self.fc(last)           # (batch, 1)
        return pred.squeeze(-1)        # (batch,)

    def count_params(self):
        return sum(p.numel() for p in self.parameters() if p.requires_grad)

# Kiểm tra
model_test = SimpleRNN(hidden_size=64, num_layers=2)
print(f"So tham số: {model_test.count_params():,}")
sample = torch.randn(4, 30, 1)      # batch=4, seq=30, feat=1
print(f"Output shape: {model_test(sample).shape}")  # (4,)
```

6 Vòng lặp huấn luyện

```
def train_model(model, Xtr, ytr, Xva, yva,
                epochs=100, lr=0.001, batch_size=64,
                clip_norm=None, verbose=True):
    """
    Huan luyen mo hinh va ghi lai lich su loss.

    Tham so:
        clip_norm : nguong gradient clipping (None = khong clip)

    Tra ve:
        history : dict chua 'train_loss' va 'val_loss' theo epoch
    """
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    dataset = torch.utils.data.TensorDataset(Xtr, ytr)
    loader = torch.utils.data.DataLoader(
        dataset, batch_size=batch_size, shuffle=False)

    history = {'train_loss': [], 'val_loss': []}

    for epoch in range(epochs):
        # --- Train ---
        model.train()
        total_loss, n_batch = 0.0, 0
        for bx, by in loader:
            optimizer.zero_grad()
            loss = criterion(model(bx), by)
            loss.backward()

            if clip_norm is not None:
                nn.utils.clip_grad_norm_(model.parameters(), clip_norm)

            optimizer.step()
            total_loss += loss.item()
            n_batch += 1

        train_loss = total_loss / n_batch

        # --- Val ---
        model.eval()
        with torch.no_grad():
            val_loss = criterion(model(Xva), yva).item()

        history['train_loss'].append(train_loss)
        history['val_loss'].append(val_loss)

        if verbose and (epoch + 1) % 20 == 0:
            print(f" Epoch {epoch+1:3d}/{epochs} | "
                  f"train={train_loss:.6f} val={val_loss:.6f}")
```

```
return history
```

```
def evaluate(model, Xte, yte, processor):  
    """Danh gia tren tap test, tra ve gia tri goc."""  
    model.eval()  
    with torch.no_grad():  
        pred = model(Xte).cpu().numpy()  
        true = yte.cpu().numpy()  
  
        pred_orig = processor.inverse(pred)  
        true_orig = processor.inverse(true)  
  
        rmse = np.sqrt(mean_squared_error(true_orig, pred_orig))  
        mae = mean_absolute_error(true_orig, pred_orig)  
        print(f" RMSE = {rmse:.3f} °C | MAE = {mae:.3f} °C")  
    return true_orig, pred_orig
```

7 Huấn luyện và vẽ đường cong học tập

```
# === HUAN LUYEN MO HINH CO BAN ===  
set_seed(42)  
model_base = SimpleRNN(hidden_size=64, num_layers=1).to(device)  
print(f"Mo hinh: {model_base.count_params():,} tham so")  
  
history = train_model(model_base, Xtr, ytr, Xva, yva,  
                      epochs=100, lr=0.001)  
  
# === VE DUONG CONG HOC TAP ===  
fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))  
  
# (a) Loss  
ep = range(1, len(history['train_loss']) + 1)  
axes[0].plot(ep, history['train_loss'], label='Train loss', color='steelblue')  
axes[0].plot(ep, history['val_loss'], label='Val loss', color='tomato',  
             ls='--')  
axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('MSE Loss')  
axes[0].set_title('Duong cong hoc tap (Loss)')  
axes[0].legend(); axes[0].grid(True, alpha=0.3)  
  
# (b) Du doan  
true, pred = evaluate(model_base, Xte, yte, processor)  
  
axes[1].plot(true, label='Thuc te', color='steelblue')  
axes[1].plot(pred, label='Du doan', color='tomato', ls='--', alpha=0.8)  
axes[1].set_xlabel('Buoc'); axes[1].set_ylabel('°C')  
axes[1].set_title('Du doan vs Thuc te (tap test)')  
axes[1].legend(); axes[1].grid(True, alpha=0.3)  
  
plt.tight_layout()  
plt.savefig('02_baseline.png', dpi=150, bbox_inches='tight')  
plt.show()
```

Cách đọc đường cong học tập:

- **Overfitting:** Train loss giảm nhưng val loss *tăng* hoặc không giảm nữa.
- **Underfitting:** Cả train loss và val loss đều cao, giảm chậm.
- **Good fit:** Cả hai giảm song song, khoảng cách nhỏ và ổn định.

Câu hỏi 4. Đường cong học tập của mô hình baseline thuộc dạng nào? Giải thích.

Phần III

Điều Chỉnh Siêu Tham Số

Trong phần này, mỗi thí nghiệm thay đổi *đúng một* siêu tham số, giữ nguyên các yếu tố còn lại. Sinh viên chạy code, ghi lại kết quả và nhận xét.

8 Thí nghiệm 1 — Hidden Size

Giả thuyết: tăng `hidden_size` cho mô hình nhiều “bộ nhớ” hơn nhưng cũng dễ overfitting hơn.

```
hidden_sizes = [16, 32, 64, 128, 256]
results_hs = {}

for hs in hidden_sizes:
    set_seed(42)
    m = SimpleRNN(hidden_size=hs, num_layers=1).to(device)
    h = train_model(m, Xtr, ytr, Xva, yva,
                    epochs=100, lr=0.001, verbose=False)
    results_hs[hs] = h
    print(f"hidden={hs:3d} | params={m.count_params():>6,} | "
          f"val_loss={h['val_loss'][-1]:.6f}")

# Ve
fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
for hs in hidden_sizes:
    axes[0].plot(results_hs[hs]['train_loss'], label=f'hs={hs}')
    axes[1].plot(results_hs[hs]['val_loss'], label=f'hs={hs}')
axes[0].set_title('Train Loss'); axes[1].set_title('Val Loss')
for ax in axes:
    ax.set_xlabel('Epoch'); ax.set_ylabel('Loss')
    ax.legend(); ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('03_hidden_size.png', dpi=150, bbox_inches='tight')
plt.show()
```

Bài tập. Điền bảng:

hidden_size	Số tham số	Val loss cuối	Nhận xét
16			
32			
64			
128			
256			

Câu hỏi 5. Giá trị nào cho val loss thấp nhất? Giá trị quá lớn gây ra vấn đề gì? (Gợi ý: so sánh khoảng cách train-val loss.)

9 Thí nghiệm 2 — Learning Rate

```
lrs = [0.01, 0.005, 0.001, 0.0005, 0.0001]
results_lr = {}

for lr in lrs:
    set_seed(42)
    m = SimpleRNN(hidden_size=64, num_layers=1).to(device)
    h = train_model(m, Xtr, ytr, Xva, yva,
                    epochs=100, lr=lr, verbose=False)
    results_lr[lr] = h
    print(f"lr={lr:.4f} | val_loss={h['val_loss'][-1]:.6f}")
```

Câu hỏi 6. LR = 0.01 có hiện tượng gì? LR = 0.0001 đã hội tụ chưa? LR nào tốt nhất?

10 Thí nghiệm 3 — Sequence Length

```
seq_lengths = [7, 14, 30, 60, 90]
results_seq = {}

for sl in seq_lengths:
    set_seed(42)
    proc_tmp = TimeSeriesProcessor(seq_len=sl)

    train_end_tmp = int(len(df) * 0.7)
    s_tr = proc_tmp.fit_transform(raw[:train_end_tmp])
    s_re = proc_tmp.transform(raw[train_end_tmp:])
    s_all = np.concatenate([s_tr, s_re])
    X_, y_ = proc_tmp.create_sequences(s_all)
    sp = proc_tmp.split(X_, y_)
    Xt, yt = to_tensor(*sp['train'])
    Xv, yv = to_tensor(*sp['val'])

    m = SimpleRNN(hidden_size=64, num_layers=1).to(device)
    h = train_model(m, Xt, yt, Xv, yv,
                    epochs=100, lr=0.001, verbose=False)
    results_seq[sl] = h
    print(f"seq_len={sl:2d} | val_loss={h['val_loss'][-1]:.6f}")
```

Câu hỏi 7. Sequence length nào tốt nhất? Tại sao quá dài có thể gây hại? (Gợi ý: liên hệ vanishing gradient ở RNN.)

11 Thí nghiệm 4 — Số lớp (Num Layers)

```
layers_list = [1, 2, 3, 4]
results_nl = {}

for nl in layers_list:
```

```

set_seed(42)
m = SimpleRNN(hidden_size=64, num_layers=nl).to(device)
h = train_model(m, Xtr, ytr, Xva, yva,
                epochs=100, lr=0.001, verbose=False)
results_nl[nl] = h
print(f"layers={nl} | params={m.count_params():>6,} | "
      f"val_loss={h['val_loss'][-1]:.6f}")

```

Câu hỏi 8. Tăng số lớp có luôn tốt hơn không? Với $\text{num_layers} \geq 3$, có dấu hiệu gì bất thường?

12 Tổng hợp kết quả siêu tham số

Sinh viên điền đầy đủ bảng sau:

Siêu tham số	Giá trị	Val loss	Nhận xét ngắn gọn
hidden_size	16		
	64		
	256		
learning_rate	0.01		
	0.001		
	0.0001		
seq_length	7		
	30		
	90		
num_layers	1		
	2		
	4		

Phần IV

Giám Sát Quá Trình Huấn Luyện Bằng TensorBoard

13 Tích hợp TensorBoard vào training loop

13.1 Nguyên lý

TensorBoard cho phép giám sát trực quan:

- **Loss/epoch** (train vs val): phát hiện overfitting.
- **Gradient norm**: phát hiện vanishing/exploding.
- **Parameter norm**: xem trọng số có “nổ” không.
- **Learning rate**: kiểm tra scheduler.

13.2 Code

```
import os

LOG_DIR = "runs/rnn_timeseries"

def train_with_tensorboard(model, Xtr, ytr, Xva, yva,
                           experiment_name="baseline",
                           epochs=100, lr=0.001, batch_size=64,
                           clip_norm=None):
    """
    Huan luyen va log day du len TensorBoard.

    Metrics duoc log:
        Loss/train, Loss/val           - phat hien overfit/underfit
        Gradient/total_norm            - phat hien vanishing/exploding
        Gradient/max_per_layer         - chi tiet tung lop
        Parameters/weight_norm         - do lon trong so
        LR                             -kiem tra scheduler
        Gap/train-minus_val            - do overfit
    """
    writer = SummaryWriter(os.path.join(LOG_DIR, experiment_name))

    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    dataset = torch.utils.data.TensorDataset(Xtr, ytr)
    loader = torch.utils.data.DataLoader(
        dataset, batch_size=batch_size, shuffle=False)

    history = {'train_loss': [], 'val_loss': []}
    step = 0

    for epoch in range(epochs):
        model.train()
        ep_loss, n = 0.0, 0

        for bx, by in loader:
            optimizer.zero_grad()
            loss = criterion(model(bx), by)
            loss.backward()

            # --- Log gradient norm MOI 5 BATCH ---
            if step % 5 == 0:
                total_norm = 0.0
                for p in model.parameters():
                    if p.grad is not None:
                        total_norm += p.grad.data.norm(2).item() ** 2
                total_norm = total_norm ** 0.5
                writer.add_scalar('Gradient/total_norm', total_norm, step)

            # Log gradient norm TUNG LOP
            for name, p in model.named_parameters():
                if p.grad is not None and 'weight' in name:
```

```

        writer.add_scalar(
            f'Gradient/{name}',
            p.grad.data.norm(2).item(), step)

    # Gradient clipping
    if clip_norm is not None:
        nn.utils.clip_grad_norm_(model.parameters(), clip_norm)

    optimizer.step()
    ep_loss += loss.item(); n += 1

    writer.add_scalar('Loss/train_step', loss.item(), step)
    step += 1

train_loss = ep_loss / n

model.eval()
with torch.no_grad():
    val_loss = criterion(model(Xva), yva).item()

history['train_loss'].append(train_loss)
history['val_loss'].append(val_loss)

# --- Log MOI EPOCH ---
writer.add_scalars('Loss/epoch',
    {'train': train_loss, 'val': val_loss}, epoch)
writer.add_scalar('Gap/train_minus_val',
    val_loss - train_loss, epoch)
writer.add_scalar('LR',
    optimizer.param_groups[0]['lr'], epoch)

# Log weight norms
for name, p in model.named_parameters():
    if 'weight' in name:
        writer.add_scalar(
            f'Parameters/{name}_norm',
            p.data.norm(2).item(), epoch)

if (epoch + 1) % 25 == 0:
    print(f"  [{experiment_name}] Epoch {epoch+1} | "
          f"train={train_loss:.6f}  val={val_loss:.6f}")

writer.flush()
writer.close()
return history

```

14 Chạy nhiều thí nghiệm với TensorBoard

```

# === Thí nghiệm A: RNN 1 lớp (baseline) ===
set_seed(42)
mA = SimpleRNN(hidden_size=64, num_layers=1).to(device)
hA = train_with_tensorboard(mA, Xtr, ytr, Xva, yva,

```



```

                                experiment_name="A_rnn_1layer",
                                epochs=100, lr=0.001)

# === Thi nghiem B: RNN 3 lop (de thay gradient issues) ===
set_seed(42)
mB = SimpleRNN(hidden_size=64, num_layers=3).to(device)
hB = train_with_tensorboard(mB, Xtr, ytr, Xva, yva,
                            experiment_name="B_rnn_3layer",
                            epochs=100, lr=0.001)

# === Thi nghiem C: RNN 3 lop + gradient clipping ===
set_seed(42)
mC = SimpleRNN(hidden_size=64, num_layers=3).to(device)
hC = train_with_tensorboard(mC, Xtr, ytr, Xva, yva,
                            experiment_name="C_rnn_3layer_clip",
                            epochs=100, lr=0.001, clip_norm=1.0)

# === Thi nghiem D: RNN lon, de overfit ===
set_seed(42)
mD = SimpleRNN(hidden_size=256, num_layers=2).to(device)
hD = train_with_tensorboard(mD, Xtr, ytr, Xva, yva,
                            experiment_name="D_rnn_big",
                            epochs=150, lr=0.001)

print("\n" + "=" * 55)
print("HOAN THANH! Mo TensorBoard de so sanh:")
print(f"  tensorboard --logdir={LOG_DIR}")
print("  Sau do mo: http://localhost:6006")
print("=" * 55)

```

15 Hướng dẫn đọc kết quả TensorBoard

```

# === HUONG DAN SU DUNG TENSORBOARD ===
#
# 1. KHOI DONG:
#   Trong terminal:  tensorboard --logdir=runs/rnn_timeseries
#   Trong Jupyter:   %load_ext tensorboard
#                   %tensorboard --logdir=runs/rnn_timeseries
#   Mo trinh duyet:  http://localhost:6006
#
# 2. TAB SCALARS - cac bieu do can xem:
#
#   Loss/epoch:
#     - So sanh train vs val cho tat ca thi nghiem
#     - Val loss tang + train loss giam => OVERFITTING
#     - Ca 2 cao va giam cham           => UNDERFITTING
#     - 2 duong sat nhau, giam deu     => GOOD FIT
#
#   Gradient/total_norm:
#     - Gia tri > 50-100                 => EXPLODING gradient
#     - Gia tri < 1e-6                   => VANISHING gradient
#     - On dinh quanh 0.01 - 10         => BINH THUONG

```

```

#         - So sanh B (khong clip) vs C (clip): thay su khac biet
#
#     Gap/train_minus_val:
#         - Tang dan theo epoch                => OVERFITTING dang xay ra
#         - Am (val < train)                    => Mo hinh chua hoc du
#
#     Parameters/weight_norm:
#         - Tang khong ngung                    => Trong so bung no
#
# 3. TIPS:
#     - Dung thanh Smoothing de lam muot duong cong
#     - Click ten run de bat/tat hien thi
#     - Tooltip hien thi gia tri chinh xac

```

Bài tập. Quan sát TensorBoard và trả lời:

1. Thí nghiệm B (3 lớp, không clip) có gradient norm bất thường không?
2. Thí nghiệm C (3 lớp + clip) khắc phục được vấn đề đó không?
3. Thí nghiệm D (model lớn) overfitting từ epoch nào? (Xem Gap/train_minus_val.)
4. Thí nghiệm nào cho val loss thấp nhất trên TensorBoard?

Phần V

Phát Hiện Và Khắc Phục Vấn Đề

16 Giám sát gradient trực tiếp

```

def train_and_record_gradients(model, Xtr, ytr, Xva, yva,
                               epochs=50, lr=0.001, clip_norm=None):
    """
    Huan luyen va ghi lai gradient norm cua TUNG LOP moi epoch.
    Dung de ve bieu do gradient flow.
    """
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    loader = torch.utils.data.DataLoader(
        torch.utils.data.TensorDataset(Xtr, ytr),
        batch_size=64, shuffle=False)

    grad_log = {name: [] for name, _ in model.named_parameters()
                 if 'weight' in name}
    history = {'val_loss': []}

    for epoch in range(epochs):
        model.train()
        epoch_grads = {k: [] for k in grad_log}

        for bx, by in loader:
            optimizer.zero_grad()
            criterion(model(bx), by).backward()

```

```

        for name, p in model.named_parameters():
            if p.grad is not None and name in epoch_grads:
                epoch_grads[name].append(
                    p.grad.data.norm(2).item())

        if clip_norm:
            nn.utils.clip_grad_norm_(model.parameters(), clip_norm)
            optimizer.step()

        for k in grad_log:
            grad_log[k].append(np.mean(epoch_grads[k]))

        model.eval()
        with torch.no_grad():
            vl = criterion(model(Xva), yva).item()
            history['val_loss'].append(vl)

    return history, grad_log

# === SO SANH: RNN 1 lop vs 3 lop ===
print("--- RNN 1 lop ---")
set_seed(42)
m1 = SimpleRNN(hidden_size=64, num_layers=1).to(device)
h1, g1 = train_and_record_gradients(m1, Xtr, ytr, Xva, yva)

print("--- RNN 3 lop ---")
set_seed(42)
m3 = SimpleRNN(hidden_size=64, num_layers=3).to(device)
h3, g3 = train_and_record_gradients(m3, Xtr, ytr, Xva, yva)

# Ve
fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
for name, vals in g1.items():
    axes[0].plot(vals, label=name.replace('rnn.', ''), alpha=0.8)
axes[0].set_title('Gradient Norm - RNN 1 lop')
axes[0].set_yscale('log'); axes[0].legend(fontsize=7)
axes[0].set_xlabel('Epoch'); axes[0].grid(True, alpha=0.3)

for name, vals in g3.items():
    axes[1].plot(vals, label=name.replace('rnn.', ''), alpha=0.8)
axes[1].set_title('Gradient Norm - RNN 3 lop')
axes[1].set_yscale('log'); axes[1].legend(fontsize=7)
axes[1].set_xlabel('Epoch'); axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('04_gradient_flow.png', dpi=150, bbox_inches='tight')
plt.show()

```

Câu hỏi 9. So sánh gradient norm giữa layer đầu và layer cuối ở mô hình 3 lớp. Có dấu hiệu vanishing không?

17 Kỹ thuật 1 — Gradient Clipping

17.1 Lý thuyết

Gradient clipping giới hạn norm của gradient:

$$g \leftarrow g \cdot \frac{\theta}{\max(\|g\|, \theta)}$$

Nếu $\|g\| \leq \theta$: giữ nguyên. Nếu $\|g\| > \theta$: co lại.

17.2 Thực hành

```
clip_values = [None, 5.0, 1.0, 0.5, 0.1]
clip_results = {}

for cv in clip_values:
    tag = f"clip={cv}" if cv else "No clip"
    set_seed(42)
    m = SimpleRNN(hidden_size=64, num_layers=3).to(device)
    h = train_model(m, Xtr, ytr, Xva, yva,
                    epochs=100, lr=0.005, # LR lon de de thay hieu ung
                    clip_norm=cv, verbose=False)
    clip_results[tag] = h
    print(f"{tag:>12} | val_loss={h['val_loss'][-1]:.6f}")

# Ve
fig, ax = plt.subplots(figsize=(8, 4.5))
for tag, h in clip_results.items():
    ax.plot(h['val_loss'], label=tag)
ax.set_title('Val Loss theo Gradient Clipping')
ax.set_xlabel('Epoch'); ax.set_ylabel('Loss')
ax.legend(); ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('05_gradient_clipping.png', dpi=150, bbox_inches='tight')
plt.show()
```

Câu hỏi 10. Giá trị clip_norm nào cho training ổn định nhất?

18 Kỹ thuật 2 — Dropout

18.1 Lý thuyết

Dropout tắt ngẫu nhiên $p\%$ neuron mỗi bước, buộc mạng học đặc trưng robust:

$$\tilde{h}_i = \begin{cases} 0 & \text{xác suất } p \\ h_i/(1-p) & \text{xác suất } 1-p \end{cases}$$

18.2 Thực hành

```
dropout_rates = [0.0, 0.1, 0.2, 0.3, 0.5]
drop_results = {}

for dr in dropout_rates:
    set_seed(42)
    m = SimpleRNN(hidden_size=128, num_layers=3, dropout=dr).to(device)
    h = train_model(m, Xtr, ytr, Xva, yva,
                    epochs=150, lr=0.001, verbose=False)
    drop_results[dr] = h
    gap = h['train_loss'][-1] - h['val_loss'][-1]
    print(f"dropout={dr:.1f} | train={h['train_loss'][-1]:.6f} | "
          f"val={h['val_loss'][-1]:.6f} | gap={abs(gap):.6f}")
```

Câu hỏi 11. Dropout = 0 có overfitting không? (So sánh gap.) Dropout nào giảm overfitting tốt nhất mà không gây underfitting?

19 Kỹ thuật 3 — Weight Decay (L2 Regularization)

19.1 Lý thuyết

Thêm penalty vào loss: $L_{\text{total}} = L_{\text{data}} + \lambda \sum w_i^2$

λ lớn $\Rightarrow$ trọng số bị ép nhỏ $\Rightarrow$ mô hình đơn giản $\Rightarrow$ giảm overfitting.

```
wds = [0, 1e-5, 1e-4, 1e-3, 1e-2]
wd_results = {}

for wd in wds:
    tag = f"wd={wd}" if wd > 0 else "No decay"
    set_seed(42)
    m = SimpleRNN(hidden_size=128, num_layers=2).to(device)
    crit = nn.MSELoss()
    opt = torch.optim.Adam(m.parameters(), lr=0.001, weight_decay=wd)
    loader = torch.utils.data.DataLoader(
        torch.utils.data.TensorDataset(Xtr, ytr), batch_size=64)

    hl = {'train_loss': [], 'val_loss': []}
    for ep in range(120):
        m.train()
        el, nb = 0, 0
        for bx, by in loader:
            opt.zero_grad()
            l = crit(m(bx), by); l.backward()
            nn.utils.clip_grad_norm_(m.parameters(), 1.0)
            opt.step(); el += l.item(); nb += 1
        m.eval()
        with torch.no_grad():
            vl = crit(m(Xva), yva).item()
        hl['train_loss'].append(el/nb)
        hl['val_loss'].append(vl)

    wd_results[tag] = hl
    print(f"{tag}>12 | val_loss={hl['val_loss'][-1]:.6f}")
```

Câu hỏi 12. Weight decay = 10^{-2} gây vấn đề gì? (Gợi ý: trọng số bị ép quá nhỏ $\Rightarrow$ underfitting.)

20 Kỹ thuật 4 — Khởi tạo trọng số

20.1 Lý thuyết

- **Xavier:** $W \sim \mathcal{U}(-\sqrt{6/(n_{\text{in}} + n_{\text{out}})}, +\dots)$ — tốt cho tanh.
- **Orthogonal:** W là ma trận trực giao — rất tốt cho RNN, giữ gradient ổn định.
- **Zeros:** $W = 0$ — rất tệ, phá vỡ tính đối xứng.

```
def apply_init(model, method):
    for name, p in model.named_parameters():
        if 'weight_ih' in name or 'weight_hh' in name:
            if method == 'xavier':
                nn.init.xavier_uniform_(p)
            elif method == 'orthogonal':
                nn.init.orthogonal_(p)
            elif method == 'zeros':
                nn.init.zeros_(p)
        elif 'bias' in name:
            nn.init.zeros_(p)

init_methods = ['default', 'xavier', 'orthogonal', 'zeros']
init_results = {}

for im in init_methods:
    set_seed(42)
    m = SimpleRNN(hidden_size=64, num_layers=2).to(device)
    if im != 'default':
        apply_init(m, im)
    h = train_model(m, Xtr, ytr, Xva, yva,
                    epochs=100, lr=0.001, verbose=False)
    init_results[im] = h
    print(f"init={im:12s} | val_loss={h['val_loss'][-1]:.6f}")
```

Câu hỏi 13. Phương pháp “zeros” cho kết quả thế nào? Giải thích. “Orthogonal” có ổn định hơn “xavier” không?

21 Kỹ thuật 5 — LayerNorm

```
class RNNWithLayerNorm(nn.Module):
    """RNN + LayerNorm trước lớp FC."""
    def __init__(self, hidden_size=64, num_layers=2):
        super().__init__()
        self.rnn = nn.RNN(1, hidden_size, num_layers, batch_first=True)
        self.norm = nn.LayerNorm(hidden_size)
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        out, _ = self.rnn(x)
        return self.fc(self.norm(out[:, -1, :])).squeeze(-1)
```

```

for use_norm, label in [(False, "RNN thường"), (True, "RNN+LayerNorm")]:
    set_seed(42)
    if use_norm:
        m = RNNWithLayerNorm(hidden_size=64, num_layers=2).to(device)
    else:
        m = SimpleRNN(hidden_size=64, num_layers=2).to(device)
    h = train_model(m, Xtr, ytr, Xva, yva,
                    epochs=100, lr=0.002, verbose=False)
    print(f"{label:16s} | val_loss={h['val_loss'][-1]:.6f}")

```

Câu hỏi 14. LayerNorm có giúp val loss ổn định hơn (ít dao động) không?

22 Bảng tổng hợp các kỹ thuật

Vấn đề	Dấu hiệu	Kỹ thuật khắc phục	Tham số cần chỉnh
Overfitting	Val loss tăng, train loss giảm; gap lớn	Dropout, Weight decay, Early stopping, Giảm model	dropout, λ , patience
Underfitting	Cả 2 loss cao, giảm chậm	Tăng hidden/layers, giảm regularization, tăng epochs	hidden_size, num_layers
Vanishing gradient	Grad norm $\rightarrow 0$ ở layer đầu	Gradient clipping, Orthogonal init, LayerNorm	max_norm
Exploding gradient	Loss = NaN, grad norm rất lớn	Gradient clipping, giảm LR	max_norm, LR

Phần VI Bài Tập

Ràng buộc chung: Mỗi lần huấn luyện ≤ 3 giờ trên máy cá nhân (GPU hoặc CPU).

23 Cấp độ 1 — Gỡ lại code và nhận xét

Mục tiêu. Sinh viên gỡ lại toàn bộ code trong bài thực hành, chạy thành công, và viết nhận xét dựa trên kết quả thu được.

23.1 Yêu cầu chi tiết

1. Thu thập dữ liệu (1 điểm):

- Gọi API Open-Meteo thành công, lấy được ≥ 700 ngày dữ liệu.
- Vẽ biểu đồ chuỗi thời gian và phân phối. Nhận xét tính mùa vụ.

2. Tiền xử lý và chia dữ liệu (1 điểm):

- Chuẩn hóa đúng cách (fit trên train, transform trên val/test).
- Chia 70/15/15 theo thứ tự thời gian. Giải thích tại sao không shuffle.

3. Huấn luyện baseline + learning curves (1.5 điểm):

- Huấn luyện RNN baseline, vẽ learning curves.
- Nhận xét: mô hình đang overfit, underfit, hay good fit?

4. Điều chỉnh siêu tham số (2 điểm):

- Chạy đủ 4 thí nghiệm (hidden, LR, seq_len, num_layers).
- Điền bảng tổng hợp. Mỗi thí nghiệm có nhận xét 1–2 câu.

5. TensorBoard (2 điểm):

- Chạy 4 thí nghiệm (A, B, C, D) với TensorBoard.
- Chụp ảnh màn hình các biểu đồ sau, kèm nhận xét 2–3 câu:
 - Loss/epoch: chỉ ra thí nghiệm nào overfit.
 - Gradient/total_norm: so sánh B (không clip) vs C (clip).
 - Gap/train_minus_val: thí nghiệm D overfitting từ epoch nào?

6. Kỹ thuật khắc phục (2.5 điểm):

- Chạy đủ 5 kỹ thuật: gradient clipping, dropout, weight decay, weight init, LayerNorm.
- Trả lời các câu hỏi trong bài.

23.2 Hình thức nộp

- File Jupyter Notebook (.ipynb) hoặc PDF.
- Code chạy được từ đầu đến cuối.
- Biểu đồ có tiêu đề, nhãn trục, chú thích.
- Nhận xét dựa trên số liệu cụ thể, không viết chung chung.

24 Cấp độ 2 — Tự code theo yêu cầu

Mục tiêu. Sinh viên tự thiết kế và cài đặt toàn bộ pipeline, chỉ dựa trên yêu cầu bên dưới, *không* copy nguyên code mẫu.

24.1 Đề bài

Dự đoán giá đóng cửa cổ phiếu bằng mô hình RNN.

24.2 Yêu cầu bắt buộc (6 điểm)

1. Dữ liệu (1 điểm):

- Dùng thư viện `yfinance` lấy dữ liệu cổ phiếu (mã tùy chọn, ≥ 2 năm).
- Thống kê mô tả và biểu đồ khám phá (giá, volume).

2. Tiền xử lý (1 điểm):

- Chuẩn hóa đúng cách (fit trên train).
- Tạo sliding window, chia train/val/test theo thời gian.

3. Mô hình (1 điểm):

- Tự code class RNN (không copy bài mẫu).
- In số tham số.

4. Huấn luyện + đánh giá (1.5 điểm):

- Training loop có gradient clipping.
- Vẽ learning curves + prediction plot.
- RMSE và MAE trên tập test.
- Nhận xét overfit/underfit.

5. Siêu tham số (1.5 điểm):

- ≥ 3 thí nghiệm thay đổi siêu tham số.
- Bảng tổng hợp + nhận xét.

24.3 Yêu cầu nâng cao (4 điểm)

1. TensorBoard (1.5 điểm):

- Tích hợp logging đầy đủ (loss, gradient norm, weight norm).
- Chạy ≥ 2 thí nghiệm khác cấu hình.
- Chụp ảnh TensorBoard + nhận xét.

2. Kỹ thuật khắc phục (1.5 điểm):

- Áp dụng ≥ 3 kỹ thuật (dropout, clipping, weight decay, init, LayerNorm, early stopping).
- So sánh trước/sau.

3. Sáng tạo (1 điểm):

- Gợi ý: dùng nhiều features đầu vào (open, high, low, close, volume); LR scheduler; early stopping tự code; so sánh RNN 1 biến vs nhiều biến; hoặc bất kỳ cải tiến nào.

24.4 Gợi ý cấu trúc code (chỉ có khung, sinh viên tự viết nội dung)

```
# === CAU TRUC GOI Y ===
```

```

# 1. LAY DU LIEU
#     import yfinance as yf
#     df = yf.Ticker("???").history(period="2y")
#     # Ve bieu do, thong ke mo ta

# 2. TIEN XU LY
#     class MyProcessor:
#         def fit_transform(self, ...): ...
#         def create_sequences(self, ...): ...
#         def split(self, ...): ...

# 3. MO HINH
#     class MyRNN(nn.Module):
#         def __init__(self, ...): ...
#         def forward(self, x): ...

# 4. TRAINING LOOP
#     def my_train(model, ...):
#         # optimizer, criterion
#         # gradient clipping
#         # log loss moi epoch
#         # (nang cao) TensorBoard writer

# 5. DANH GIA
#     # Learning curves
#     # Prediction plot
#     # RMSE, MAE

# 6. SIEU THAM SO
#     # >= 3 thi nghiem
#     # Bang tong hop

# 7. KY THUAT KHAC PHUC (nang cao)
#     # Dropout, clipping, weight decay, init, LayerNorm

```

24.5 Quy định

- Hạn nộp: 2 tuần từ ngày giao.
- Code chạy được, không lỗi.
- **Cấm copy** nguyên văn từ bài mẫu hoặc bạn khác.
- Mỗi lần train ≤ 3 giờ.

Phần VII

Tài Liệu Tham Khảo

1. Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press. Ch.10: Sequence Modeling.
2. Karpathy, A. (2015). The Unreasonable Effectiveness of Recurrent Neural Networks. Blog.

3. Pascanu, R., Mikolov, T., Bengio, Y. (2013). On the difficulty of training recurrent neural networks. *ICML*.
4. Smith, L. (2018). Cyclical Learning Rates for Training Neural Networks. *WACV*.
5. PyTorch `nn.RNN`: pytorch.org/docs/stable/generated/torch.nn.RNN.html
6. TensorBoard for PyTorch: pytorch.org/docs/stable/tensorboard.html
7. Open-Meteo API: open-meteo.com/en/docs
8. Yahoo Finance (`yfinance`): pypi.org/project/yfinance